

深度学习作业 5

221900180 田永铭 tianym2022@smail.nju.edu.cn

2024 年 12 月 23 日

Problem 1 列举几种常见的激活函数，并简要说明它们的作用。

Solution: 解：

1. Sigmoid 函数

公式： $\sigma(x) = \frac{1}{1+e^{-x}}$

作用：将输入值映射到 0 和 1 之间。适用于二分类问题的输出层，因为其输出可解释为概率。但在深层网络中容易导致梯度消失问题，因此不常用于隐藏层。

2. Tanh（双曲正切）函数

公式： $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

作用：输出值在 -1 和 1 之间，比 Sigmoid 有更好的性能，因为其输出是零中心的，有助于加速梯度下降。与 Sigmoid 类似，Tanh 也存在梯度消失的问题。

3. ReLU（Rectified Linear Unit）

公式： $\text{ReLU}(x) = \max(0, x)$

作用：当输入大于 0 时，输出为输入值；否则输出为 0。ReLU 是目前最常用的激活函数之一，能够有效缓解梯度消失问题，且计算简单。其缺点是会出现“死神经元”问题，导致部分神经元永远不会激活。

4. Leaky ReLU

公式： $\text{Leaky ReLU}(x) = \max(\alpha x, x)$ （其中 α 是一个小常数）

作用：Leaky ReLU 是对 ReLU 的改进，允许输入小于 0 时输出为一个很小的负数，避免了死神经元问题。通常， α 取值较小，如 0.01。

5. GELU (Gaussian Error Linear Unit)

公式： $\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3) \right) \right)$

作用：GELU 是一种平滑的非线性激活函数，常用于 Transformer 等模型中。它结合了高斯误差函数和线性激活的优点，使得网络在训练时能够获得更好的梯度流，避免 ReLU 的死神经元问题，并具有类似于 ReLU 的高效性。

Problem 2 什么是反向传播？它的核心思想是什么？

Solution: 解:

反向传播 (Backpropagation) :

是一种用于训练神经网络的算法，它通过链式法则有效地计算损失函数关于网络中每个参数的梯度，从而实现模型的优化。反向传播通过梯度下降算法来更新神经网络中的权重和偏置，使得损失函数的值逐渐减小，从而提高网络的性能。

核心思想:

反向传播的核心思想是利用链式法则，从输出层开始，逐层将误差反向传播到隐藏层和输入层，计算每个参数对损失函数的偏导数，并利用这些梯度更新参数。

Problem 3 Numpy 实现反向传播梯度。

```
1 import numpy as np
2
3 # 设置随机种子，方便复现
4 np.random.seed(42)
5
6 # 初始化参数
7 W1 = np.random.randn(2, 2) # 输入层到隐藏层的权重
8 b1 = np.random.randn(2)    # 隐藏层的偏置
9 W2 = np.random.randn(2)    # 隐藏层到输出层的权重
10 b2 = np.random.randn()     # 输出层的偏置
11
12 # 输入数据和真实标签
13 x = np.array([1.0, -1.5])
14 y_true = 0.5
15
16 # 学习率
17 learning_rate = 0.01
18
19
20 # ReLU
21 def relu(x):
22     #todo 输入x 返回relu(x)
23     pass
24
25
26 # ReLU 的梯度
27 def relu_derivative(x):
28     return np.where(x > 0, 1, 0)
29
```

```
30
31 # 均方损失函数和其导数
32 def mse_loss(y_pred, y_true):
33     #todo 计算均方误差
34     pass
35
36
37 def mse_loss_derivative(y_pred, y_true):
38     return y_pred - y_true # 对 y_pred 求导
39
40
41 # 训练过程
42 epochs = 100
43 for epoch in range(epochs):
44     # 前向传播
45
46     z1 = np.dot(W1, x) + b1
47     h = relu(z1)
48     y_pred = np.dot(W2, h) + b2
49
50     # 计算损失
51     loss = mse_loss(y_pred, y_true)
52
53     # 反向传播
54     d_loss_y_pred = mse_loss_derivative(y_pred, y_true) # 损失对预测值的梯度
55     #todo
56     #d_loss_W2 = ? 对 W2 的梯度
57     #d_loss_b2 = ? 对 b2 的梯度
58
59     #d_loss_h =? 损失对隐藏层输出 h 的梯度
60     #d_loss_z1 =? 隐藏层的梯度 (通过 ReLU 计算)
61
62     #d_loss_W1 =? 对 W1 的梯度
63     #d_loss_b1 =? 对 b1 的梯度
64
65     # 参数更新 (随机梯度下降)
66     W1 -= learning_rate * d_loss_W1
67     b1 -= learning_rate * d_loss_b1
68     W2 -= learning_rate * d_loss_W2
69     b2 -= learning_rate * d_loss_b2
70
71     # 打印训练过程中的损失
```

```
72     if epoch % 10 == 0:
73         print(f"Epoch {epoch}: Loss = {loss}")
74
75     # 打印最终的损失值
76     print(f"Final Loss: {loss}")
```

Solution: 补充部分代码如下:

```
1
2 # ReLU
3 def relu(x):
4     #todo 输入x 返回relu(x)
5     return np.maximum(0, x)
6
7
8 def mse_loss(y_pred, y_true):
9     #todo 计算均方误差
10    return np.mean((y_pred - y_true) ** 2) / 2
11
12
13 # 反向传播
14 d_loss_y_pred = mse_loss_derivative(y_pred, y_true) # 损失对预测值的梯度
15 # d_loss_W2: 损失对 W2 的梯度
16 d_loss_W2 = d_loss_y_pred * h.T # 对 W2 求导, 链式法则, h 是列向量, 使用 h.
    T 进行矩阵乘法
17 # d_loss_b2: 损失对 b2 的梯度
18 d_loss_b2 = d_loss_y_pred # 对 b2 求导
19 # d_loss_h: 损失对隐藏层输出 h 的梯度
20 d_loss_h = d_loss_y_pred * W2.T # 反向传播到隐藏层
21 # d_loss_z1: 隐藏层的梯度 (通过 ReLU 计算)
22 d_loss_z1 = d_loss_h * relu_derivative(z1) # 通过 ReLU 的导数来计算
23 # d_loss_W1: 损失对 W1 的梯度
24 d_loss_W1 = np.outer(d_loss_z1, x) # 对 W1 求导, 链式法则, x 是输入
25 # d_loss_b1: 损失对 b1 的梯度
26 d_loss_b1 = d_loss_z1 # 对 b1 求导
```

其它求导步骤都很简单, 利用链式法则即可, 对 $W1$ 的梯度, 由于 $W1$ 是一个矩阵, 所以求导比较复杂, 代码 24 行具体解释如图 1。(注意这里应该每一步都有转置和每一步都没有出来是一样的, 本质相同。)

The image shows handwritten notes on a piece of paper. On the left, there is a diagram of a simple neural network with two input nodes (\$x_1, x_2\$), two hidden nodes (\$z_1, z_2\$), and one output node (\$y\$). Weights are labeled \$w_{11}, w_{12}, w_{21}, w_{22}\$ and biases are \$b_1, b_2\$. On the right, there are equations for the forward pass: \$z_1 = w_{11}x_1 + w_{21}x_2 + b_1\$, \$z_2 = w_{12}x_1 + w_{22}x_2 + b_2\$, and \$y = \text{relu}(z_1)\$. Below the diagram, there are several mathematical derivations for the backpropagation of gradients. The first derivation shows the gradient of the loss \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The second derivation shows the gradient of \$L\$ with respect to the hidden node outputs \$z_1\$ and \$z_2\$. The third derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The fourth derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The fifth derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The sixth derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The seventh derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The eighth derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The ninth derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector. The tenth derivation shows the gradient of \$L\$ with respect to the weights \$w_{11}\$ and \$w_{12}\$ as a vector.

图 1

以上是全写成标量计算的，其实也能用正常方法， x 在这里是数组形状 $[x_1, x_2]$ ， dz 也是 $[dz_1, dz_2]$ ，只要计算 $x^T dz$ (矩阵乘法)，也能得到一样的结果。

代码运行结果如图 2，可以看到损失逐渐降低，最终很小，说明训练有效。

```
Epoch 0: Loss = 0.025835351192849892
Epoch 10: Loss = 0.0020375628244268906
Epoch 20: Loss = 0.0001564739167320906
Epoch 30: Loss = 1.1920868579827328e-05
Epoch 40: Loss = 9.061439375630744e-07
Epoch 50: Loss = 6.883600510263259e-08
Epoch 60: Loss = 5.228286352452871e-09
Epoch 70: Loss = 3.9708406640054e-10
Epoch 80: Loss = 3.015781510371962e-11
Epoch 90: Loss = 2.2904231367201775e-12
Final Loss: 2.2510213012190517e-13
```

图 2: tarin_loss

Problem 4 什么是全连接网络（FC）或多层感知机（MLP）？它与线性分类器的主要区别是什么？

Solution: 解：

全连接网络（FC）或多层感知机（MLP）

全连接网络（FC）或多层感知机（MLP）是一种神经网络模型，由多层神经元组成，其中每一层的每个神经元与前一层的每个神经元都有连接。它是最基础的神经网络类型之一，广泛用于分类和回归任务。

- **输入层：**接收原始输入数据。
- **隐藏层：**通过加权和偏置进行计算，并通过激活函数进行非线性变换。
- **输出层：**根据任务需求输出结果，如分类标签或回归值。

MLP 是一种前馈神经网络，信息在网络中单向传播，不会有反馈连接。每一层的输出都成为下一层的输入。

线性分类器

线性分类器（如线性回归或支持向量机）通过输入特征的线性组合来做出预测，输出一个线性函数，通常表示为：

$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

其中 $x_1, x_2, \dots, x_n$ 是输入特征， $w_1, w_2, \dots, w_n$ 是对应的权重， b 是偏置项。

而多层感知机通过多个层级和非线性激活函数的组合，可以拟合更复杂的非线性关系。每一层的输出通过激活函数进行非线性变换，从而具有更强的表达能力。

主要区别

- **线性与非线性：**
 - 线性分类器只能表示输入特征的线性关系。
 - MLP 使用非线性激活函数，使其能够学习复杂的非线性关系。
- **层次结构：**
 - 线性分类器只有一个层次，直接从输入到输出。
 - MLP 由多层组成，包括至少一个隐藏层。
- **表达能力：**
 - 线性分类器的表达能力较弱，一般仅适用于线性可分的任务。
 - MLP 可以逼近任何复杂的函数，适用于更广泛的任务。

Problem 5 在梯度流的基本计算模式中，加法门 (add gate)、乘法门 (mul gate) 和最大值门 (max gate) 分别具有什么特点？

Solution: 解：

加法门：加法门不改变梯度，只是将梯度传递到所有输入。

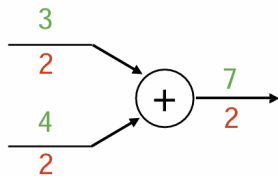
乘法门：乘法门会将梯度乘以其他输入的乘积（假设是多个，如果是两个输入的乘法则是另一个输入）传到这个输入。

最大值门：最大值门会将梯度传递给选择出的最大值，但如果最大值有多个，则平分梯度。

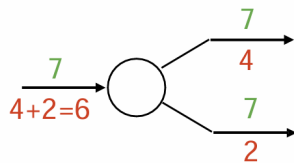
直观展示见图 3。

■ 梯度流中的基本计算模式

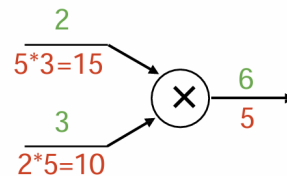
add gate: gradient distributor



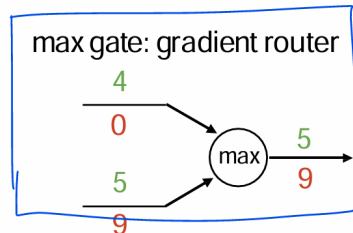
copy gate: gradient adder



mul gate: "swap multiplier"



max gate: gradient router



81

图 3